

ANALYSIS OF ALGORITHM

IA-2

MERGESORT

VISUALISER

NAME & ROLL NO.

Batch: C-3

Om Bhanushali :- 16010123215

Om Thanage :- 16010123217

Omik Acharya:- 16010123218

Introduction

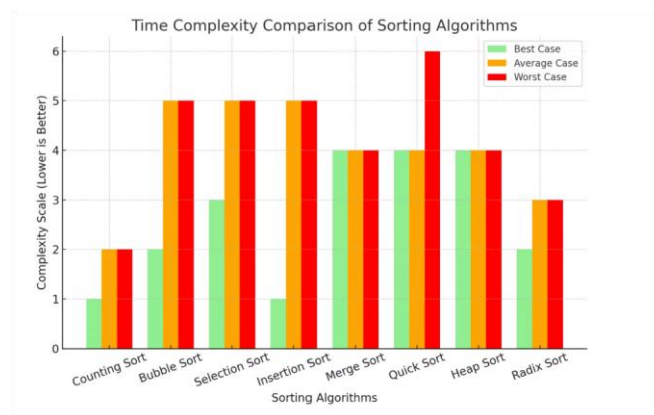
What is merge sort?

Merge Sort is a classic divide-and-conquer sorting algorithm that divides an array into halves, recursively sorts each half, and then merges the sorted halves together. It's known for its **$O(n \log n)$** time complexity and **stable sorting behavior**.

Analysis

Merge sort is a stable, efficient algorithm with $O(n \log n)$ time complexity in all cases (average/worst), solving the recurrence $T(n) = 2T(n/2) + n$. Its worst-case comparisons range between $n \log n - n + 1$ and $n \log n + n$, with the best case using ~50% fewer steps. On average, it saves $\alpha \cdot n$ comparisons ($\alpha \approx 0.2645$) versus the worst case.

Compared to quicksort, merge sort uses 39% fewer comparisons in its worst case than quicksort's average, while matching quicksort's best-case move complexity ($O(n \log n)$). Though not in-place (requiring $O(n)$ extra space, reducible to $n/2$), it excels with sequential data access, making it popular in languages like Lisp.



MergeSort Visualizer

[\[Visualizer Link\]](#)

[\[Video Demo\]](#)

Merge Sort Visualizer is an **interactive web tool** that shows how the **Merge Sort** algorithm works step-by-step using animations. Instead of just printing numbers in a console, it visually represents an array as a series of bars where the height of each bar corresponds to the value of the element.

As the algorithm sorts the array, the bars change positions, colours, and heights, letting users see how the array is being divided, compared, and merged back together in a sorted order.

Why use a visualizer?

- **Educational:** It helps learners understand how the merge sort algorithm works behind the scenes by breaking down each step.
- **Visual Learning:** Recursion and the merging process become easier to understand when represented visually.
- **Debugging:** A visualizer allows users to observe the algorithm's behaviour in real-time, making it easier to analyse and improve performance.

How does it work?

1. Array Initialization and Input Handling

When the webpage is loaded, users can initialize the array in one of several ways:

- Manually enter numbers in the input field and click **Apply**.
- Use predefined options: **Random**, **Sorted**, or **Reversed**.

Each of these actions triggers specific event handlers which utilize the following functions:

- `parse()` – Parses user input to form an array.
- `generateRandomArray()`, `generateSortedArray()` – Generate arrays based on the selected pattern.
- `displayArray()` – Renders the array visually as blocks within the animation area.

These functions work in tandem to prepare and display the array prior to sorting.

2. Starting the Sorting Process

Clicking the **Sort** button initiates the merge sort animation process. The following steps are executed:

- The button's click event handler retrieves the current visual array by calling `getArrayValue()`.
- The array is then passed to the asynchronous function `sort(arr)`, which begins the sorting logic and handles the animation sequence.

3. Divide Phase: Recursive Splitting of the Array

Within the `sort(arr)` function:

- The base case is evaluated to determine whether the array is already sorted or small enough to stop recursion.
- If not, the array is divided into two subarrays using `createSubArray(arr, from, to)`, producing `half1` and `half2`.
- These subarrays are visually animated to separate from each other using `animateDivision(half, dir)`.
- Each division step is documented via `addStep(description)`, which appends a description to the on-screen list of operations.
- Recursion continues by calling `await sort()` on each half independently.

4. Merge Phase: Reconstructing the Sorted Array

Once both halves have been recursively sorted, they are merged back into the parent array:

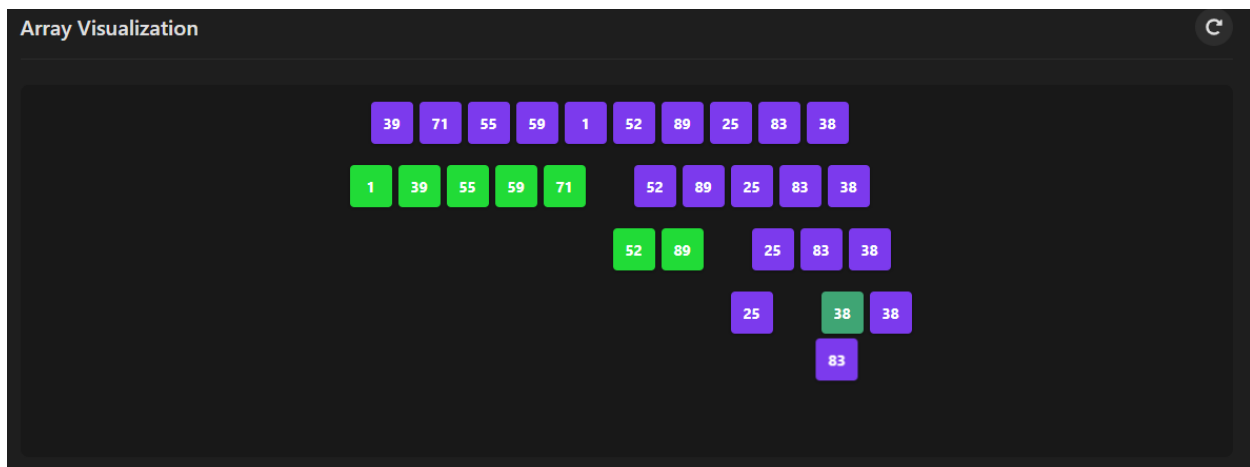
- The function `merge(arr1, arr2, target)` is called, where `arr1` and `arr2` are the sorted subarrays, and `target` is the original container array.
- Elements from both subarrays are compared.
- The smaller element is moved to its correct position using the animated function `await animateMergeAlgorithmPlacement(element, target)`.
- Each merge step is also described using `addStep(description)` to maintain transparency of operations.

5. Unsorting the Array

The **Unsort** button provides a way to reverse the order of the current array:

- The event handler reads the existing array values.
- It reverses their order.
- The updated array is passed to `unsort(arr)`, which redraws the blocks in reversed order via `displayArray()`.

This feature allows users to test sorting behaviour from a different starting point.



Features

Custom Array Configuration

- **Manual Number Entry**

Users have the ability to input their own comma-separated list of numbers. This feature empowers users to test specific scenarios, edge cases, or personally curated datasets.

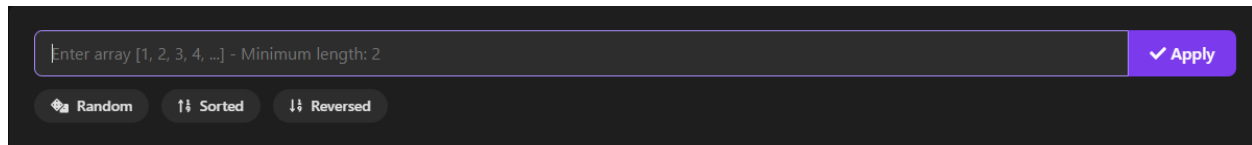
Instant Array Generation

- **Predefined Array Modes**

For quick setup and comparative analysis, the interface offers a set of chip-style buttons that instantly generate arrays in three distinct patterns:

- **Random** – A shuffled selection of values.
- **Sorted** – An ascending array.
- **Reversed** – A descending array.

These features streamline the process of exploring merge sort by giving users control over the input structure without requiring manual data entry every time.

A screenshot of a user interface for array configuration. It features a dark background with a light gray input field containing the placeholder text "Enter array [1, 2, 3, 4, ...] - Minimum length: 2". To the right of the input field is a purple button with a white checkmark and the text "Apply". Below the input field are three chip-style buttons: "Random" with a shuffle icon, "Sorted" with an upward arrow icon, and "Reversed" with a downward arrow icon.

Animated Sorting:

- **Dynamic Array Containers**

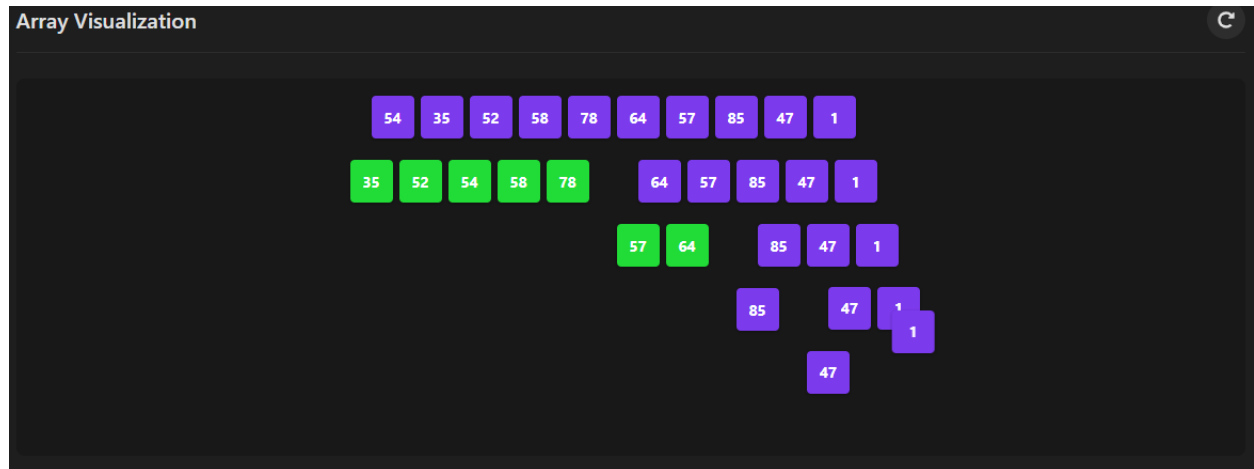
Arrays are displayed as visually distinct groups that move and divide across the screen, helping users clearly see the recursive nature of merge sort.

- **Color-Coded Number Blocks**

Each number in the array appears as a coloured block, making it easy to distinguish values at a glance and observe their movement throughout the sorting process.

- **Smooth Transitions and Highlights**

As numbers are compared, moved, and merged, they animate fluidly. Subtle transitions in position and colour highlight the algorithm's logic and decisions, making each step easy to follow.



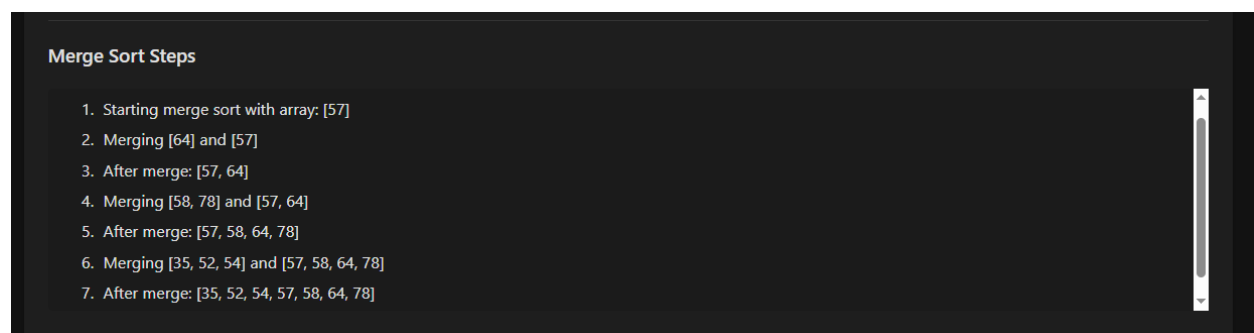
Step-by-Step Log

To complement the visual animation, the tool provides a running textual log that narrates the sorting process in real time.

- **Descriptive Sorting Timeline**

Each split and merge operation is recorded as a brief, readable description, helping users follow the algorithm's logic step by step.

The log is housed in a dedicated panel, allowing users to review the entire sequence of actions at their own pace.



How to Use the Merge Sort Visualizer

1. Input the Array

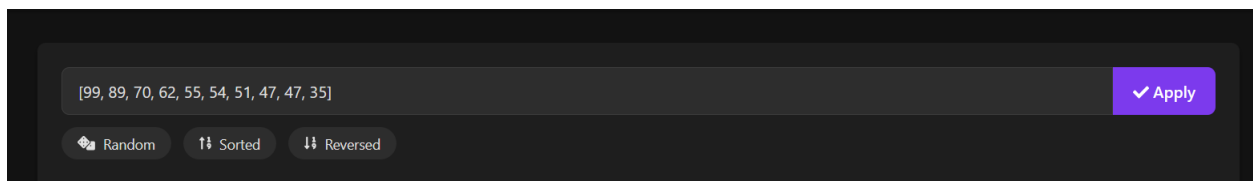
- At the top of the interface, there is an input field pre-filled with a sample array.
- You may modify this array by entering your own comma-separated list of integers (e.g., 23, 11, 45, 78).
- Once you've entered your desired array, click the Apply button to confirm your input.

2. Generate a Preset Array

Below the input section, three preset array generation options are available:

- Random: Generates a random array of integers.
- Sorted: Fills the array with integers in ascending order.
- Reversed: Fills the array with integers in descending order.

Use these buttons to quickly test different initial configurations.

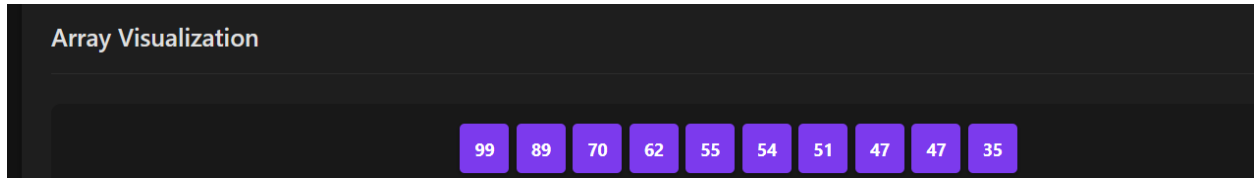


3. Visualize the Array

- After clicking Apply, the array is displayed in the Array Visualization section as colored blocks representing each element.
- These blocks will dynamically update during the sorting process to demonstrate each stage of the algorithm.

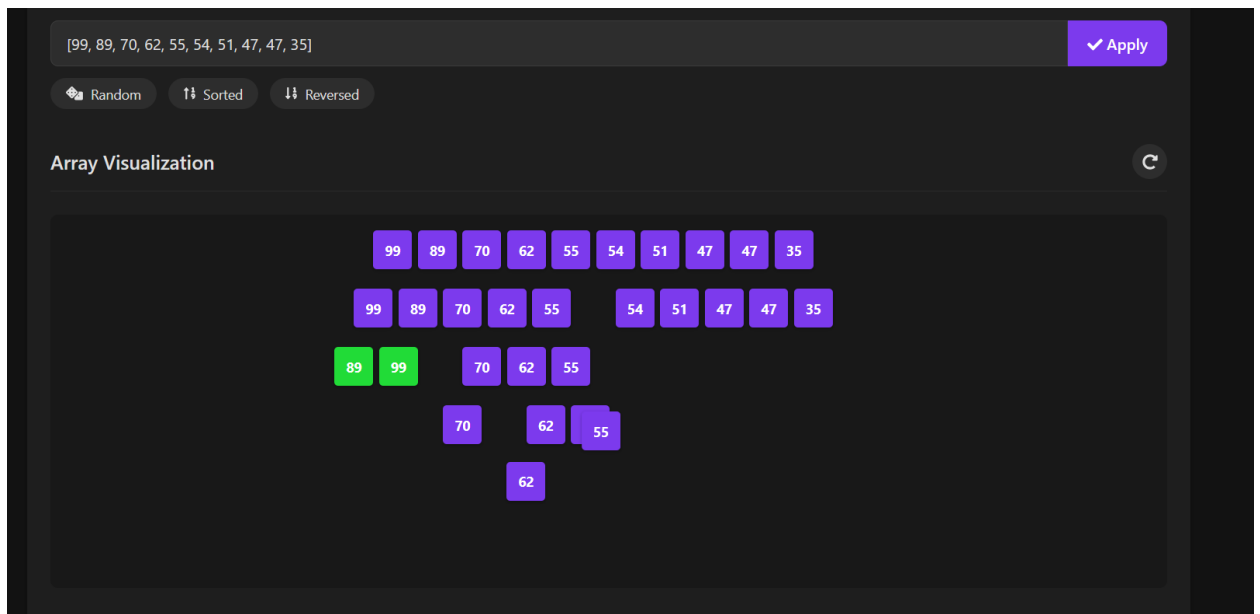
4. Observe the Sorting Process

- Click the Sort button at the bottom of the page to initiate the Merge Sort visualization.
- As the sorting progresses, the visualization panel shows the division and merging phases typical of Merge Sort.



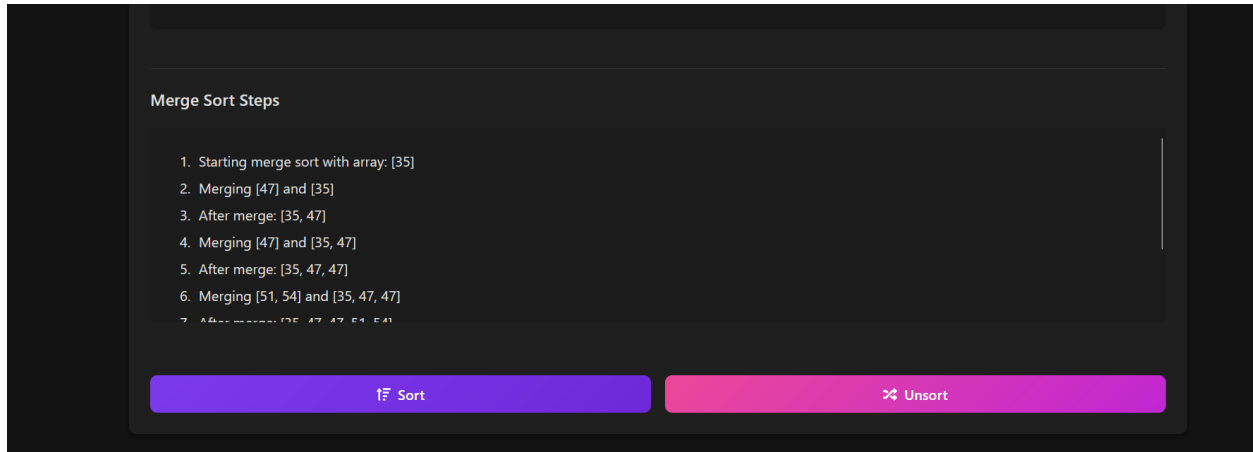
5. Track Step-by-Step Execution

- Below the visualization panel, the Merge Sort Steps section provides a textual description of each action performed during the sort.
- This includes information about how the array is being split and how subarrays are being merged.



6. Reset the Visualization

- To return the array to its unsorted state, click the Unsort button.
- This allows you to rerun the sorting process or try a different configuration.



Conclusion

The Merge Sort Visualizer provides a clear and structured way to understand the merge sort algorithm through visual representation. It breaks down each step of the sorting process, highlighting how the array is recursively divided and merged.

With features such as customizable input, preset array configurations, and step-by-step tracking, the tool is intended to support learning and debugging by offering better insight into how merge sort operates internally. It serves as a useful aid for students and developers looking to explore the logic of merge sort beyond code execution.

Video link: <https://www.youtube.com/watch?v=KzHNjCs1bgw>

Hosted website: <https://aoamergesort.vercel.app/>